Block chain di Bitcoin

Protocollo

L'architettura di Bitcoin si basa su un modello permision-less

Il protocollo di Bitcoin fa in modo che valgano:

1. Eventual consistency:

   Tutti i nodi del sistema presentano lo stesso vettore dei blocchi, eccetto qualche delle ultime entry

2. Liveness: → La versione di Bitcoin di Validity

   Quando un nodo onesto riceve una transazione T_x, prima o poi viene inserita all'interno di un blocco

3. Un meccanismo per consentire ad un nuovo nodo di determinare il vettore corretto

Sybyl Attacks

In un sistema Permisionless è possibile che un nodo corrotto ne simuli un numero arbitrario da immettere nel modello

Funzione Hash crittografica

Una funzione Hash crittografica è una funzione

$$H: \{0,1\}^* \longrightarrow \{0,1\}^\ell$$ → In Bitcoin si usa SHA256 dove $\ell = 256$

Per cui devono valere:

1. Preimmage Resistance:

   Dato $y \in \{0,1\}^\ell$ è difficile trovare $x \in \{0,1\}^*$ t.c. $H(x) = y$

2. Second preimmage Resistance:

   Dato $x_1 \in \{0,1\}^*$ è difficile trovare $x_2 \in \{0,1\}^*$ t.c. $H(x_1) = H(x_2)$

3. Collision resistance: → Non è possibile trovare x senza Brute Force

   È difficile trovare $x_1, x_2 \in \{0,1\}^*$ t.c. $x_1 \neq x_2$ e $H(x_1) \neq H(x_2)$

Composizione del blocco

Ogni nodo del sistema presenta un vettore di blocchi, dove ogni blocco è composto da

- Block Header

  Contenente le informazioni che identificano un blocco contenuti in 80 byte divisi così:

  - Version: numero di versione
  - Prev-Hash: l'Hash del blocco precedente
  - Data-Hash: l'Hash del contenuto del blocco
  - Timestamp: data e ora in cui viene creato il blocco
  - Target: un'intero fondamentale per la proof of work
  - Nonce: un'intero necessario per la proof of work

- Block Data

  Insieme delle transazioni

Proof of Work

Il protocollo di Bitcoin utilizza la Proof of Work per far valere l'Eventual Consistency e la Liveness

La logica dietro la Proof of Work è semplice:

Siano:

- $t \leq 2^{256}$ il Target su cui tutti i nodi si mettono d'accordo
- blk è il nuovo blocco da inserire con header blk.header
- h = Hash(blk.header) l'hash del blocco

Allora

Un blocco è valido $\iff h \leq t \wedge$ blk.prev.hash = hash(header del blocco precedente)

↳ Stiamo dicendo che, modificando il nonce, bisogna trovare un input t.c. l'Hash dell'header sia inferiore al Target

Criterio di scelta del Target

Il Target viene scelto in modo tale che venga scelto un nuovo blocco ogni 10 minuti

Il ricalcolo del Target avviene ogni 2016 blk per adattarsi alla potenza di calcolo della rete

↳ 14 giorni

Biforcazione della catena

Può capitare che due blk validi e differenti fra loro, A e B vengano aggiunti quasi contemporaneamente

In questo caso si creano due catene identiche tranne che per l'ultimo blocco

Quando un nodo riceve entrambe le versioni, le mantiene entrambe, ma estenderà quella con più lavoro computazionale

Alla fine la rete converge sulla catena con più proof of work cumulativa

↳ la catena più lunga